OPERATING SYSTEM

Complete Study Notes

OS Basics · Functions · Techniques of OS · CPU Scheduling Algorithms · Deadlock · PCB & Process States · Schedulers · Interrupts

Typed from the handwritten class notes — every topic, example and diagram retained, with every scheduling example worked out step by step.

Contents

1. Operating System — Basics 3

1.1 Types of OS interface (by number of users) . . . 3

1.2 Parts of the OS . . . 3

1.3 Interface of the OS . . . 3

2. Functions of the OS 3

3. Techniques (Types) of Operating Systems 5

4. CPU Process Scheduling Algorithms 7

4.1 FIFO / FCFS — First Come First Served . . . 7

4.2 SJF — Shortest Job First . . . 7

4.3 Round Robin (RR) . . . 8

4.4 Priority scheduling . . . 9

4.5 SRTF — Shortest Remaining Time First . . . 9

4.6 MQS — Multilevel Queue Scheduling . . . 10

4.7 MLFQ — Multilevel Feedback Queue Scheduling . . . 10

4.8 HRRN — Highest Response Ratio Next . . . 10

4.9 FSS — Fair Share Scheduling . . . 11

4.10 Comparison of the algorithms . . . 11

5. Deadlock 12

Four conditions for deadlock . . . 12

Deadlock handling methods . . . 12

6. PCB — Process Control Block 12

Process states . . . 13

Context switching . . . 14

Schedulers . . . 14

7. Interrupts 14

Types of interrupt . . . 15

Appendix — Clarifications and reading notes 16

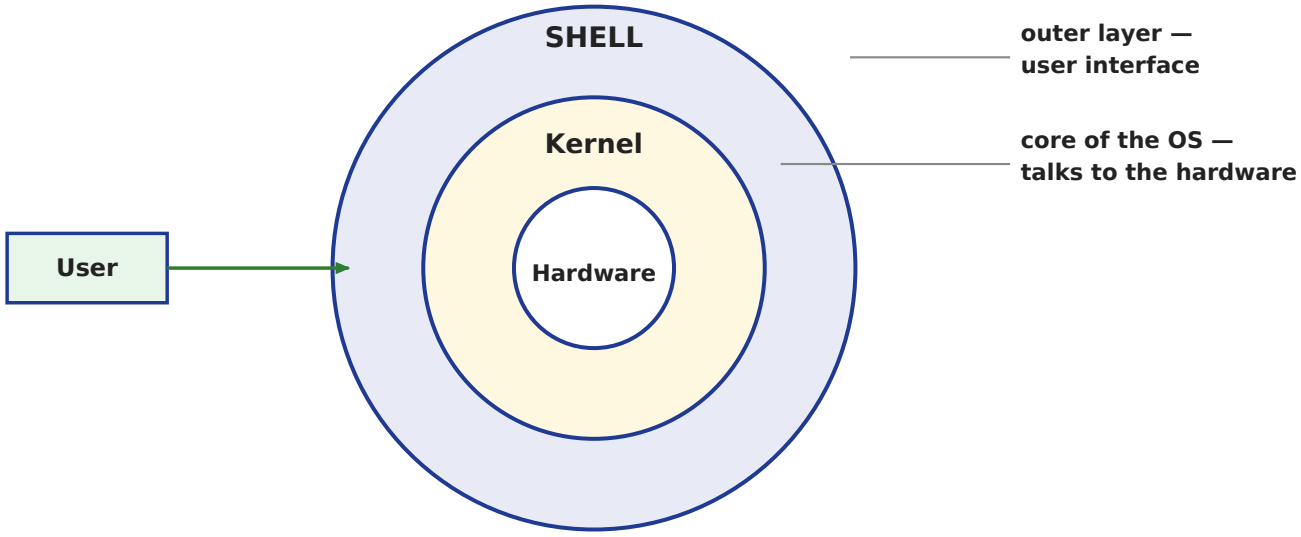
1. Operating System — Basics

- The OS is a **link between the user and the hardware**.
- It acts as a **bridge** to enable communication between the user and hardware, and between the user and application software.

1.1 Types of OS interface (by number of users)

Type	Meaning	Examples
(1) SUI — Single User Interface	An interface where **only one user** can work at a time	Android, MS-DOS, iOS etc.
(2) MUI — Multiple User Interface	An interface where **more than one user** can work at the same time	Linux, Unix etc.

1.2 Parts of the OS



Shell (outer layer), kernel (core) and hardware.

Part	Meaning
(a) Shell (outer layer)	The user interfaces with this layer
(b) Kernel	The **internal layer** of the OS; interacts with the hardware; the **core** of the OS

1.3 Interface of the OS

Interface	Details	Examples
(i) CUI / CLI character user interface / command line interface	To perform any task, the related **command must be typed**	MS-DOS, Unix etc.
(ii) GUI graphical user interface	The related **graphical tool** is used. For an OS to be considered a GUI it must follow 4 conditions — **WIMP**: Window, Icon, Menus, Pointer	Android, MS Windows, iOS, Macintosh

2. Functions of the OS

(1) Process management

- The main task of this module is to **create and delete processes**.

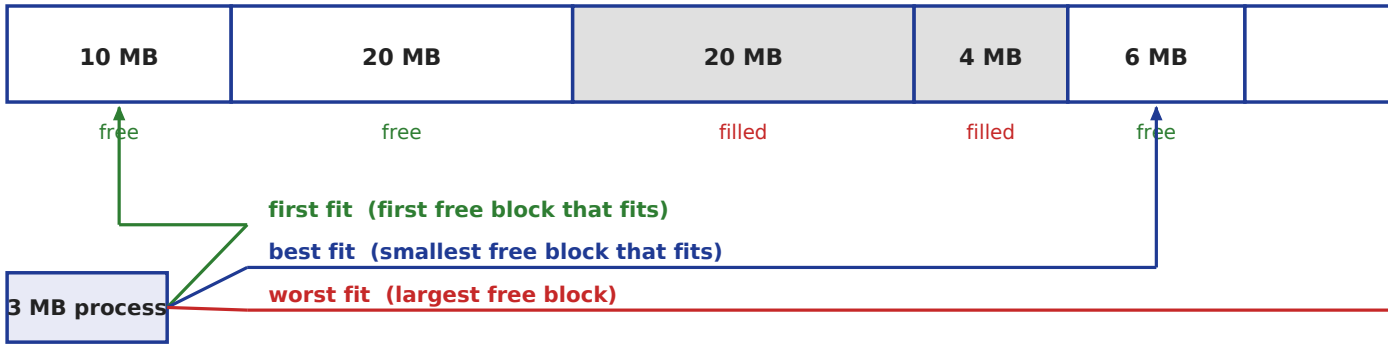
- The OS **schedules CPU time** for all available processes in memory — called **process scheduling** (or just scheduling).

(2) File management

- The OS module **creates, deletes and retrieves files**.
- e.g. **Rashi.txt** — file name + extension.
- File systems: FAT, NTFS, APFS, HFS, exFAT, ... ZFS.

(3) Memory management

- The OS is responsible for memory **allocation and deallocation**.
- Allocation strategies: **best fit, first fit, worst fit**.



A 3 MB process placed in a memory of free and filled blocks by first fit, best fit and worst fit.

Strategy	Chooses	Block used for the 3 MB process
First fit	The **first** free block that is big enough	10 MB (the first free block)
Best fit	The **smallest** free block that is big enough	6 MB (4 MB is filled, 6 MB is the tightest free fit)
Worst fit	The **largest** free block	30 MB

Note

The block sizes in the handwritten picture (10, 20, 20, 4, 6, 30, 8 MB, with three blocks marked “filled”) are hard to read; the arrows above follow the standard meaning of first, best and worst fit on that picture.

(4) Security management

- The OS is responsible for the **security** of the computer system and for **preventing unauthorized access**.
- Passwords are used for files, folders and system entry.

(5) Input–output management

- Through its module the OS manages **input–output devices and operations**.

(6) Integrated management

- Data is managed by the OS in an **integrated form**, so the working of one data does not disturb the processing time of another (the notes say “extra processing time”).

(7) Command management

- The OS **responds to the command / instruction** given by the user.

(8) Disk management

- The OS manages the **secondary memory / storage / hard disk** of the computer system.

(9) Error management

- The OS manages the **errors** occurring in the computer system.
- e.g. when the printer runs out of paper.

3. Techniques (Types) of Operating Systems

(1) Batch processing OS

- A **serial OS**: instead of processing the same type of jobs separately, they are processed together in a **single batch**.
- There is **no direct connectivity** between the user and the system, so it is called an **offline** processing system.
- Used in **mainframes** or where large data is handled.
- All tasks / jobs are processed **sequentially**.

(2) Multitasking OS

- The OS can handle **multiple tasks at the same time**.
- While the OS is executing one task it can allocate resources to another task — called **time slicing**.
- e.g. Android, Windows.
- CPU time is divided among the tasks.

(3) Multiprogramming OS

- **Multiple programs are loaded into memory** at the same time, but **only one program uses the CPU at a time**.
- Minimizes the CPU idle time.
- e.g. Linux OS.
- Because of multiprogramming, multitasking is possible.

(4) Multiprocessing

- When a system uses **multiple CPUs** to process tasks simultaneously.

Type	Meaning
(a) Symmetric	All CPUs have **equal access**
(b) Asymmetric	One CPU is **primary**, the others are secondary

(5) RTOS — Real-Time OS

- Manages **real-time applications**; **time-critical** and must be completed within a specific timeframe.

Feature	Hard RTOS	Soft RTOS
Deadline	Strict deadline	Flexible deadline
Impact of a missed deadline	System fails	System does not fail; only performance is affected
Examples	Medical devices, aviation, nuclear	Live streaming, online gaming
Response time	Deterministic	Non-deterministic
Use case	Systems that are time-critical	Not time-critical

(6) Time-sharing OS

- A single processor / CPU is **shared among multiple processes**.
- The CPU is allocated to all processes for a short time, called a **time slice** or **quantum time**.
- Each process executes during its time slice; as soon as the slice ends it goes to the **ready** state and waits for its next time slice.

(7) Distributed OS

- **Multiple independent computers** are connected / integrated to work as a **single system**.

- e.g. Facebook, Instagram, Google.

4. CPU Process Scheduling Algorithms

Formulas

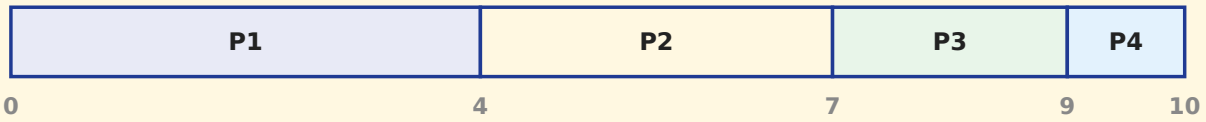
Turn Around Time (TAT) = Completion time (CT) − Arrival time (AT)
Waiting Time (WT) = TAT − Burst time (BT)
Burst time = time taken by a process to complete, i.e. its execution time.
Response time = time when the process got the CPU − time when the process entered memory.

4.1 FIFO / FCFS — First Come First Served

- Processes are served in the order they arrive.

Example: FCFS — four processes

Process	Arrival time	Burst time
P1	0	4
P2	1	3
P3	2	2
P4	3	1



Process	AT	BT	Start	Finish (CT)	TAT = CT − AT	WT = TAT − BT
P1	0	4	0	4	4	0
P2	1	3	4	7	6	3
P3	2	2	7	9	7	5
P4	3	1	9	10	7	6

Average WT = (0 + 3 + 5 + 6) / 4 = 14/4 = **3.5**; Average TAT = (4 + 6 + 7 + 7) / 4 = 24/4 = **6**.

4.2 SJF — Shortest Job First

- The process with the **shortest burst time** is executed first.
- Can be preemptive or non-preemptive (the notes write “preemptive (SJN) or non-preemptive (SJF)”; the preemptive form is SRTF, section 4.5).
- SJF **minimizes the average waiting time**.
- **Starvation** can occur: if small processes keep coming, the longer processes may never get executed.

Example: SJF — all processes arrive at 0

Process	Arrival time	Burst time
P1	0	3
P2	0	2
P3	0	4



Process	AT	BT	Start	Finish (CT)	TAT	WT
P2	0	2	0	2	2	0
P1	0	3	2	5	5	2
P3	0	4	5	9	9	5

TAT = CT − AT, WT = TAT − BT. Average WT = 7/3 = **2.33**; Average TAT = 16/3 = **5.33**.

4.3 Round Robin (RR)

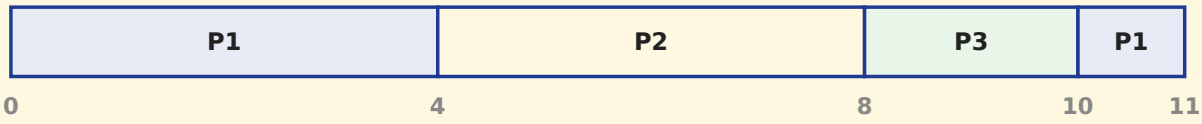
- Each process is allocated a **fixed time slot** of the CPU — a **time slice** or **quantum time**.
- If a process does not finish in its time slice, it waits in the queue for the next turn.
- The slice is so small that all processes feel like only they are being processed.

Advantages	Disadvantages
Fair scheduling No starvation Best for time-sharing systems	Time-quantum selection is tough: • too small → **context-switching overhead** • too large → **works like FCFS**

Example: Round Robin — time slice = 4 ms

Process	Arrival time	Burst time
P1	0	5
P2	0	4
P3	0	2

Find: (A) average TAT, (B) average WT, (C) completion time of P2.



Process	AT	BT	Completion time	TAT	WT
P1	0	5	11	11	6
P2	0	4	8	8	4
P3	0	2	10	10	8
Total				29	18

(A) Average TAT = 29/3 ≈ **9.67** (B) Average WT = 18/3 = **6** (C) Completion time of P2 = **8**.

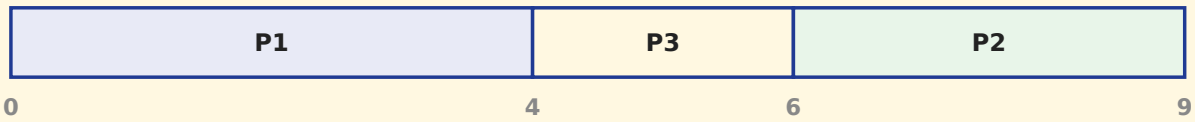
How it runs: P1 uses 4 ms (1 ms left) → P2 uses 4 ms and finishes at 8 → P3 uses 2 ms and finishes at 10 → P1 gets the CPU again for its last 1 ms and finishes at 11.

4.4 Priority scheduling

- Each process is given a **priority number** as input; processes with **higher priority are executed first**.
- If processes have the **same priority**, it works like **FCFS**.
- If the priority of a process changes, it is called preemption (as written in the notes).
- Efficient for real-time use.
- **Starvation** can occur, as lower-priority processes may never get executed.

Example: Priority scheduling (a lower priority number runs first)

Process	AT	BT	Priority
P1	0	4	2
P2	2	3	3
P3	4	2	1



Process	AT	BT	Priority	Start (ST)	CT	TAT	WT
P1	0	4	2	0	4	4	0
P3	4	2	1	4	6	2	0
P2	2	3	3	6	9	7	4

Average TAT = (4 + 2 + 7) / 3 = **13/3**; Average WT = (0 + 0 + 4) / 3 = **4/3**. When P1 ends at 4, P3 (priority 1) has arrived and beats P2 (priority 3).

Note

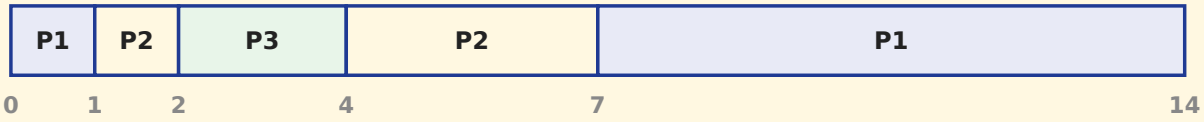
The “Priority” column heading is hard to read in the handwritten table; the values 2, 3, 1 are consistent with the order P1 → P3 → P2 that the notes show.

4.5 SRTF — Shortest Remaining Time First

- The **preemptive version of SJF**.
- The CPU is given to the process with the **shortest remaining burst time**.
- If a new process arrives with a remaining time **less than the current process**, the current process is **interrupted**.
- Reduces the average waiting time; **starvation is possible**.

Example: SRTF — completion time of P1

Process	Arrival time	Burst time
P1	0	8
P2	1	4
P3	2	2



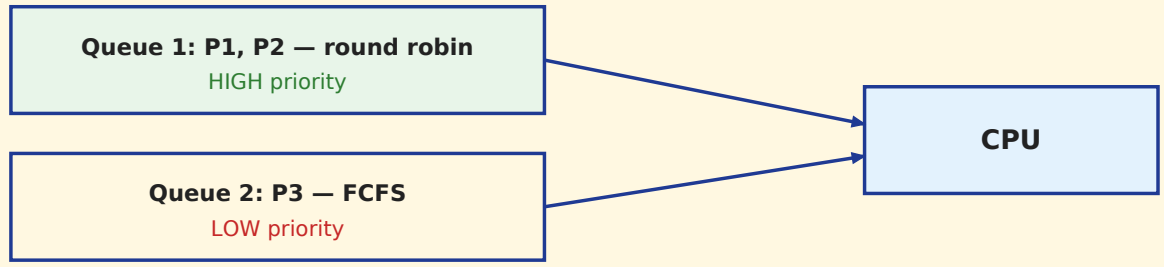
Time	What happens
0 – 1	Only P1 is there → runs (7 ms left)
1 – 2	P2 arrives with 4 < 7 → P1 is interrupted; P2 runs (3 ms left)
2 – 4	P3 arrives with 2 < 3 → P2 is interrupted; P3 runs and finishes at 4
4 – 7	P2 (3 left) is shorter than P1 (7 left) → P2 finishes at 7
7 – 14	P1 runs its remaining 7 ms and finishes at 14

Completion time of P1 = 14. (Added: TAT of P1, P2, P3 = 14, 6, 2; WT = 6, 2, 0; average TAT = 22/3 ≈ 7.33, average WT = 8/3 ≈ 2.67.)

4.6 MQS — Multilevel Queue Scheduling

- Processes are divided into **different queues** (based on priority / burst time), and **each queue uses a different scheduling algorithm**.
- Difficult to manage and implement.

Example: Two queues



execution order: P1 → P2 → P3

Queue 1 = P1 & P2 (round robin) = high priority; Queue 2 = P3 (FCFS) = low priority; execution order: **P1 → P2 → P3**.

4.7 MLFQ — Multilevel Feedback Queue Scheduling

- An **advanced version of MQS**.
- A process's queue **can change based on its behaviour**.
- A process that uses **more CPU = low priority**; a process that **gives up the CPU for I/O quickly = high priority**.
- Used in **modern OS** (Linux, Windows).
- No starvation; but very complex and hard to tune the parameters.

4.8 HRRN — Highest Response Ratio Next

- **Non-preemptive**; used to **avoid starvation**.
- Response ratio = **(burst time + waiting time) ÷ burst time**.

- The process with the **highest response ratio** is executed first (a waiting process's ratio keeps growing, so it cannot starve).
- Drawback: **calculation overhead**.

4.9 FSS — Fair Share Scheduling

- The CPU is divided **among users, not among processes**.
- Each user gets an **equal CPU share**, no matter how many processes they have.
- **User-based** scheduling; used in multi-user systems.

Example: Two users

User 1 has 3 processes and User 2 has 10 processes — the CPU allotment is still **50 : 50** between the two users.

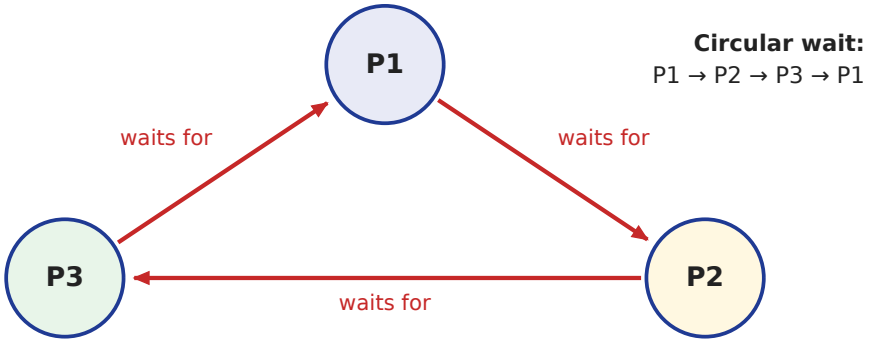
4.10 Comparison of the algorithms

Algorithm	Preemption	Starvation	Complexity
FCFS	✗	✗	L (low)
SJF	✗	✓	M (medium)
SRTF	✓	✓	H (high)
RR	✓	✗	M
Priority (PS)	✓ / ✗	✓	M
MQS	✓ / ✗	✓	H
MLFQ	✓	✗	VH (very high)
HRRN	✗	✗	M
FSS	✓	✗	VH

✓ = yes, ✗ = no. In the handwritten table the FCFS starvation cell is faint; FCFS does not starve processes, so ✗ is shown.

5. Deadlock

- When **two or more processes hold some resources** (RAM, scanner, disk etc.) and **demand other resources that are held by other processes**, an infinite wait condition is created — this is called **deadlock**.



Circular wait: P1 waits for P2, P2 waits for P3 and P3 waits for P1 (picture added from the example in the notes).

Four conditions for deadlock

#	Condition	Meaning
(i)	**Mutual exclusion**	At least one resource must be held in **non-sharable mode** — only one process can use the resource at any given time
(ii)	**Hold and wait**	A process is **holding at least one resource** and is **waiting for other resources** held by other processes
(iii)	**No preemption**	A resource held by a process **cannot be forcibly taken away**; it is released only voluntarily after the process finishes
(iv)	**Circular wait**	A closed chain of processes exists, each waiting for a resource held by the next one — e.g. P1 → P2 → P3 → P1

Deadlock handling methods

Method	How it works
(1) Deadlock prevention	Make **at least one of the 4 conditions fail**. If circular wait is stopped, deadlock does not occur
(2) Deadlock avoidance	Before allocating a resource to any process, the OS checks whether the system will go into an **unsafe state**; uses the **Banker's algorithm** to check for safe and unsafe states
(3) Deadlock detection & recovery	Deadlock is detected using a **cycle-detection algorithm**; recovery by **killing / terminating the process** or by **preempting (forcibly taking) the resources**
(4) Deadlock ignorance	**Ostrich algorithm**; mostly used in Windows & Linux. It is assumed that deadlock will almost never occur, and if it is detected the system is simply restarted

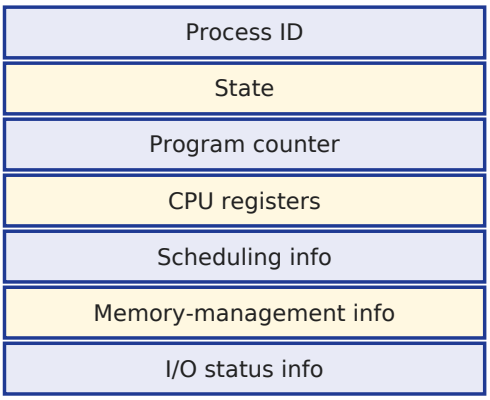
Remember

Memory aid from the notes: **Resource → Process → Avoid, Prevent**.

6. PCB — Process Control Block

- Every process has a **PCB** — its **ID card / resume** — which is maintained by the OS.

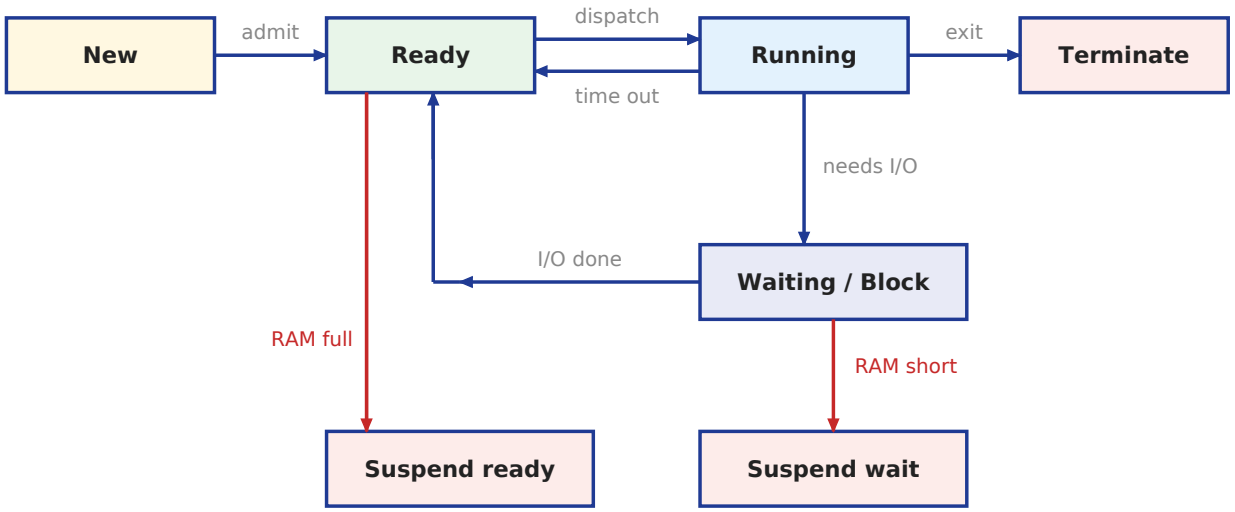
PCB (process control block)



Part	Holds
(a) Process ID	Unique number of the process
(b) State	New, ready, running, waiting …
(c) Program counter	Address of the next instruction
(d) CPU registers	Saved register values
(e) Scheduling info	Priority, queue pointers
(f) Memory-management info	Memory used by the process
(g) I/O status info	Devices / files in use

The right-hand column explains each part briefly (added); the notes list only the seven names.

Process states



Suspended processes are shifted to the hard disk (picture added from the list of states)

Process-state diagram: new → ready ↔ running → terminate, with waiting and the two suspended states.

#	State	Meaning
(1)	**New**	The process is **created for the first time**; not yet in main memory
(2)	**Ready**	The process is **loaded into RAM** and is waiting to get the CPU. Multiple processes can be in the queue; the **scheduler** decides which one gets the CPU next
(3)	**Running**	The CPU is **executing the instructions** of the process
(4)	**Waiting / Block**	If the process needs an **I/O operation** it leaves the CPU and goes into the waiting state; once the I/O work is done it returns to the ready state
(5)	**Terminate / Exit**	When the process is fully executed the OS **terminates it and removes it from memory**
(6)	**Suspend wait**	The process is in the waiting state and there is a **shortage of RAM** → it is shifted to the hard disk
(7)	**Suspend ready**	The ready queue (RAM) is **full** → the process is shifted to the hard disk

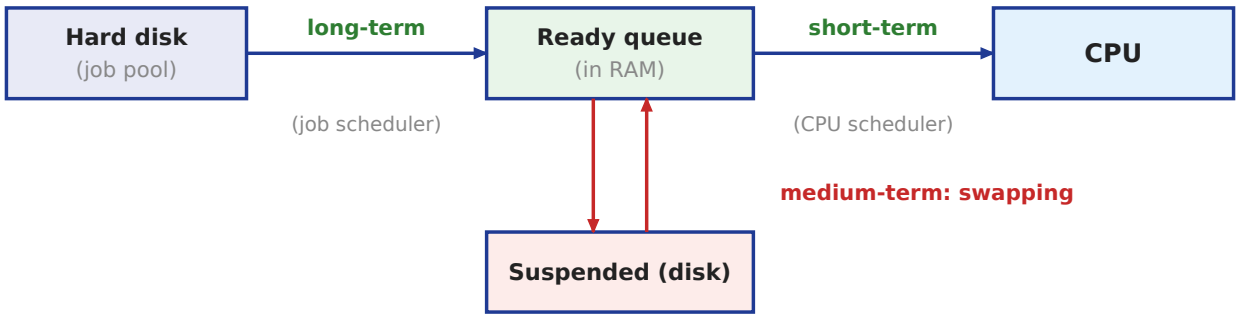
Note from the page

When RAM becomes full, the OS shifts some processes to the hard disk (this is what the two “suspend” states describe).

Context switching

- The CPU **switches from one process to another**.
- While doing so, when the CPU leaves process P1 it **saves its current details in its PCB** (so P1 can later resume exactly where it stopped).

Schedulers

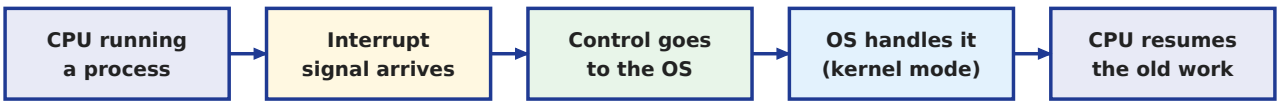


Long-term, short-term and medium-term schedulers (picture added from the descriptions in the notes).

Scheduler	Also called	Job
(i) Long term	Job scheduler	Decides which process will be **brought from the hard disk to RAM**
(ii) Short term	CPU scheduler — the **fastest**	Decides which process goes from the **ready state to the running state**
(iii) Medium term	—	Responsible for **swapping**: if RAM is full it sends processes to the secondary memory

7. Interrupts

- An interrupt is a **signal / event**.
- It **tells the CPU to stop the current work** and do some other urgent work first.
- By default **control goes to the OS** and the OS handles the interrupt.
- Interrupts are always handled in **privileged mode / kernel mode**.



What happens when an interrupt arrives (flow added from the points in the notes).

Types of interrupt

	(A) Hardware interrupt	(B) Software interrupt
Generated by	An **external device**	The **program itself**
Comes from	Outside the CPU	Inside the running program
Used for / sources	Keyboard, printer etc.	To access OS services — on **read(), write(), open()**
Timing	**Asynchronous** — can occur at any time	**Synchronous** — occurs after an instruction
Predictable?	No	**Predictable**
Example	Key press, printer ready	**System call**

Appendix — Clarifications and reading notes

The handwritten pages were typed as faithfully as possible. These places were clarified, flagged or marked as added:

Topic	In the handwritten notes	Here
Memory-fit picture	Block sizes and “filled” marks hard to read	Followed the standard meaning of first / best / worst fit; flagged
SJF	“preemptive (SJN) or non-preemptive (SJF)”	Kept; preemptive form is SRTF
Priority scheduling	“If the priority of a process changes, it is called preemption”	Kept as written; in standard usage preemption means interrupting a running process when a higher-priority one arrives
Priority example	Priority column heading hard to read	Values 2, 3, 1 match the order shown; flagged
Round robin	Averages written as 29/3 and 18/3	Also shown as ≈ 9.67 and 6
FCFS starvation	Cell faint in the comparison table	Shown as ✗ (FCFS does not starve)
Process states	Numbered (1)–(3), then lettered D–G	Numbered (1)–(7) in order
Added for revision	—	SRTF per-process TAT/WT, process-state graph, deadlock cycle, scheduler and interrupt flow pictures, PCB part descriptions